

Knowledge Transfer (KT) Document: Data Table

Download/Print Functionality

Overview

This document provides a detailed breakdown of the testing approach used to verify PDF download functionality, specifically handling the case where clicking a "Save PDF" button triggers the browser's native print dialog.

1. Which Approach Are We Using?

We are using the **CDP (Chrome DevTools Protocol) Print-to-PDF & `window.print` Mocking Approach**.

Instead of interacting with the native print dialog, we overwrite (mock) the browser's `window.print` method. When the application tries to print, our mock catches the call and silently commands Chrome (via CDP) to generate a real, text-searchable PDF file in the background. Finally, we read the generated file and parse its text to assert its contents.

2. Why Are We Using This Approach?

- **Native OS Dialogs are Untestable:** Playwright (and other web automation tools) cannot interact with native Operating System dialogs, such as the Print or Save As dialogs triggered by `window.print()`. If the test simply clicks the button, the OS dialog will pop up, and the test will hang indefinitely waiting for user input.
- **True PDF Verification:** Unlike simply checking if a button was clicked, this approach actually generates a physical PDF file that mimics what the user would print. We can then read the file to ensure the application sends the correct data to the print view.
- **Official Workaround:** This is the recommended and official Playwright workaround for handling `window.print()` functionality.

3. Inch-by-Inch Breakdown (Step-by-Step)

Here is exactly what is happening in the code, step-by-step:

Imports & Setup

```
import { test, expect } from '@playwright/test';
import fs from 'fs';
const pdfParse = require('pdf-parse');
```

- `fs` : Node.js File System module, used to write and read the generated PDF file on the local disk.
- `pdf-parse` : A library used to extract text from a physical PDF file so we can perform assertions on the content.

Browser Restriction

```
const isChromiumBased = ['chromium', 'edge', ''].includes(testInfo.project.name);
test.skip(!isChromiumBased, 'CDP PDF generation is only supported in Chromium-based browsers');
```

- **Why?** The Chrome DevTools Protocol (`Page.printToPDF`) is exclusive to Chromium-based browsers

(Chrome, Edge). The test skips execution gracefully if run in Firefox or WebKit.

Navigation

```
await page.goto('http://localhost:5173');
await page.waitForSelector('.data-table');
```

- Navigates to the local application and waits for the `.data-table` to be fully loaded in the DOM before proceeding.

1. Setting up the CDP Session

```
const client = await page.context().newCDPSession(page);
```

- Opens a direct communication channel to the underlying Chrome browser engine, allowing us to send low-level instructions (like printing).

2. Exposing the Node.js Function

```
await page.exposeFunction('triggerCDPPrint', async () => { ... });
```

- `page.exposeFunction` makes a Node.js function available *inside* the browser's JavaScript context (window object). This bridges the gap between the browser and our backend test script.

Inside `triggerCDPPrint` :

1. `await page.emulateMedia({ media: 'print' });` -> Forces the page to apply `@media print` CSS styles so the PDF looks like a printed page.
2. `await new Promise(r => setTimeout(r, 100));` -> A brief pause to ensure the browser has fully applied the print CSS before capturing the PDF.
3. `client.send('Page.printToPDF', { ... })` -> Instructs Chrome to generate the PDF with exact A4 dimensions and background graphics.
4. `await page.emulateMedia({ media: 'screen', colorScheme: 'no-preference' });` -> Reverts the page back to its normal screen state.
5. `fs.writeFileSync(pdfPath, Buffer.from(data, 'base64'));` -> Decodes the base64 PDF data returned by Chrome and saves it as a `.pdf` file on the local disk.
6. `pdfGenerated = true;` -> Flips a boolean flag so our test knows the async PDF generation is complete.

4. Mocking `window.print`

```
await page.evaluate(() => {
  window.print = () => {
    window['triggerCDPPrint']();
  };
});
```

- **Crucial Step:** We inject JavaScript into the page to overwrite the default `window.print` function. Now, when the app calls `window.print()`, it will NOT open the OS dialog. Instead, it will call our exposed `triggerCDPPrint` function.

Triggering the Action

```
const printButton = page.getByRole('button', { name: 'Save PDF' });
await printButton.click();
```

- We simulate the user clicking the "Save PDF" button, which internally calls `window.print()` , thereby triggering our CDP PDF generation.

5. Waiting for the PDF

```
await expect.poll(() => pdfGenerated).toBe(true);
```

- Because PDF generation is asynchronous, Playwright polls the `pdfGenerated` flag repeatedly until it turns `true` , ensuring we don't proceed before the file is fully saved to disk.

6. Validating the Physical File

```
expect(fs.existsSync(pdfPath)).toBe(true);
const stats = fs.statSync(pdfPath);
expect(stats.size).toBeGreaterThan(0);
```

- We check the local file system to confirm the file was actually created and is not empty (size > 0 bytes).

7. Validating the PDF Content

```
const dataBuffer = fs.readFileSync(pdfPath);
const data = await pdfParse(dataBuffer);

const text = data.text;
expect(text).toContain('Global Data Overview');
// ... other assertions ...
```

- Finally, we read the PDF file back into memory, parse it using `pdf-parse` , and extract its raw text.
- We run assertions against the text to guarantee that the final generated PDF contains the expected data (`ENTITY NAME` , `TOTAL AMOUNT` , etc.).